



University
of Glasgow

**Friday 3 May 2024
14:00 – 15:30 BST
Duration: 1 hour 30 minutes
Timed exam – fixed start time**

DEGREE OF MSc

**Deep Learning MSc (M)
COMPSCI5103**

(RUBRIC: Answer All Questions)

**This examination paper is an open book, online
assessment and is worth a total of 60 marks.**

Instead of spending a long time using the Moodle maths editor, you should try to format mathematical results using text (for instance using NumPy expressions for vectors and matrices). For instance, you could write `"[[dy/dw1, dy/dw2], [dy/dw3, dy/dw4]]"` to represent:

$$\begin{bmatrix} \frac{\partial y}{\partial w_1} & \frac{\partial y}{\partial w_2} \\ \frac{\partial y}{\partial w_3} & \frac{\partial y}{\partial w_4} \end{bmatrix}$$

1. You should answer the following questions by manually examining the following PyTorch model.

```
class Net(nn.Module):
    def __init__(self, value1 = 15, value2 = 7):
        super(Net, self).__init__()
        self.conv1 = nn.Conv1d(20, 5, value1, padding = value1//2)
        self.conv2 = nn.Conv1d(5, 3, value2, padding = value2//2)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        x = self.conv2(x)
        return x

loss = nn.CrossEntropyLoss()
```

- (a) What is the general name given to this type of deep neural network and what is unique about its input and output tensors compared to a conventional ConvNet? [2]
- (b) How many input and output channels does the model have? Suggest what this model could be used for. [2]
- (c) What are the `value1` and `value2` being used to decide? Explain what process within the machine learning pipeline is facilitated by writing the model class like this. [2]
- (d) A tensor is input to this network that has shape: `torch.size([1, 20, 50])`. Give the shape of the tensors output by the two convolutional layers. [4]
- (e) Produce a single formula for the number of parameters in this network in terms of the variables `value1` and `value2`. Describe key steps of working for full marks. [7]
- (f) Explain whether the output of this network is appropriate to use with the `CrossEntropyLoss` shown or whether something is missing in the model. Describe the shape and datatype of tensor that would be required as the ground truth target. [3]

**Kernel of length 3;
2 channels**

Input Tensor

Channel 1:

1	0	1
---	---	---

Channel 1:

1	2	3	4	5
---	---	---	---	---

Channel 2:

0	1	0
---	---	---

Channel 2:

6	7	8	9	10
---	---	---	---	----

Figure 1: 1D convolution and input tensor.

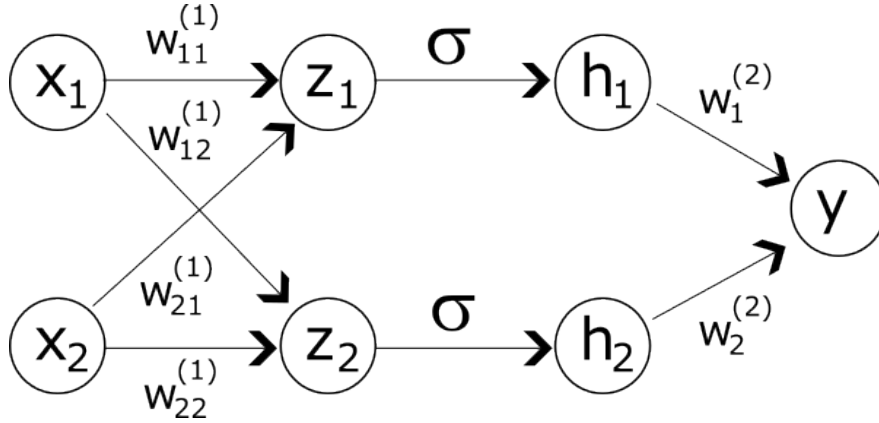


Figure 2: Fully connected neural network.

2. (a) Figure 1 shows a one dimensional convolutional kernel of length 3 with 2 channels. Calculate the result of applying this kernel to the input tensor shown. Assume no padding and a stride of 1. Show appropriate working. [3]
- (b) If max pooling(length = 3, stride = 2) was applied to the Input Tensor in Figure 1, what would be the output? [3]
- (c) Figure 2 shows a neural network using a sigmoid activation function using notation that should be familiar to you. Let $\mathbf{x} \in \mathbb{R}^2$, $\mathbf{z} \in \mathbb{R}^2$, $\mathbf{h} \in \mathbb{R}^2$ denote the relevant vectors. Let $\mathbf{M} \in \mathbb{R}^{2 \times 2}$ and $\mathbf{N} \in \mathbb{R}^{1 \times 2}$ denote relevant matrices.
In matrix notation, write out the equation for y as a function of $\mathbf{h}$ using the relevant matrix. In addition, write out the elements of the matrix in terms of symbols given in the figure. [4]
- (d) From Figure 2, write out the matrix equation for y as a function of $\mathbf{x}$ using the relevant matrices and operations. [3]
- (e) Calculate the gradient vector $\nabla_{\mathbf{h}} y$ in terms of variables given in Figure 2. [4]
- (f) By using back-propagation, determine elements of the gradient vector: $\nabla_{\mathbf{z}} y$ [3]

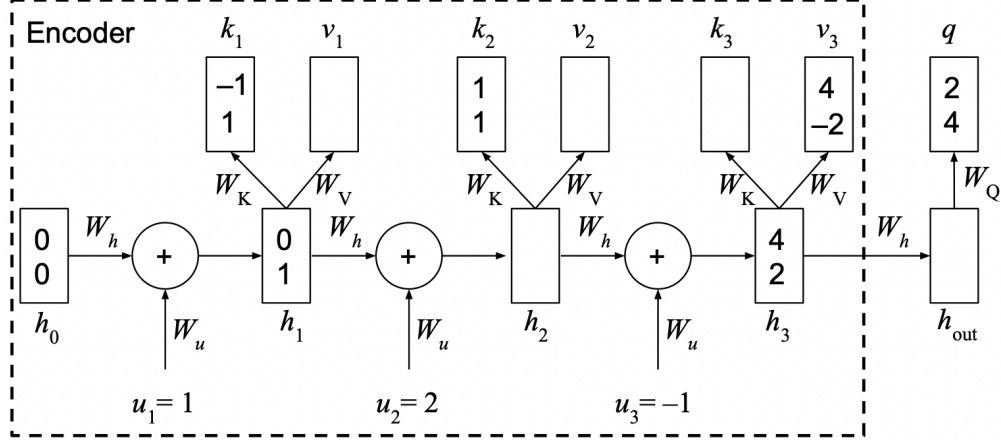


Figure 3: RNN Encoder.

3. Consider a linear RNN with a two-dimensional state h_t and a scalar input u_t at each timestep:

$$\mathbf{h}_t = W_h \mathbf{h}_{t-1} + W_u u_t$$

Suppose $W_h = \begin{bmatrix} -1 & 2 \\ 0 & 1 \end{bmatrix}$, $W_u = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$, and we initialize $\mathbf{h}_0 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$. The inputs are $u_1 = 1, u_2 = 2, u_3 = -1$. The decoder state is $\mathbf{h}_{\text{out}} = W_h \mathbf{h}_3$. We also generate the keys, values, and queries for decoder-side cross-attention as purely linear functions of the state using weight matrices:

$$W_K = \begin{bmatrix} 2 & -1 \\ -1 & 1 \end{bmatrix}$$

$$W_V = \begin{bmatrix} 1 & 0 \\ -1 & 1 \end{bmatrix}$$

$$W_Q = \begin{bmatrix} 3 & 1 \\ 4 & 2 \end{bmatrix}$$

A partially filled in computation graph is provided in Figure 3. The edges with weights W on them indicate that the vector is left-multiplied by that weight matrix as it passes along that edge.

- Provide the missing values for the blank boxes in Figure 3 ($v_1, h_2, v_2, k_3, h_{\text{out}}$). [5]
- What would be the output of attention for the decoder's query? To simplify calculations, use an argmax instead of softmax. For example, $\text{softmax}([1, 3, 2])$ becomes $\text{argmax}([1, 3, 2]) = [0, 1, 0]$. Show key steps of working and final result. [5]
- What is the role of Attention in Linear RNNs, and how does it differ from traditional RNNs without Attention?. [4]
- Explain the computation of attention scores and the generation of context vectors. [2]

- (e) Provide one real-world example to illustrate the impact of Attention and briefly explain why the Attention is particularly advantageous in this scenario. [4]